



Center for Geographic Analysis
Institute for Quantitative Social Science
Harvard University

Title: Using Python for Geospatial Analysis and Visualization

Instructors: Jielu Zhang and Devika Jain

Time: 9:00 AM–12:00 PM, May 22, 2026

Location: 1737 Cambridge St., Cambridge, MA, CGIS Knafel Building, Room SB-12, Sub-Basement

Zoom: <https://harvard.zoom.us/j/2304706732?omn=92517624341>

Pre-Workshop Instructions

Laptops are required for this workshop. Please complete the following steps before the workshop begins:

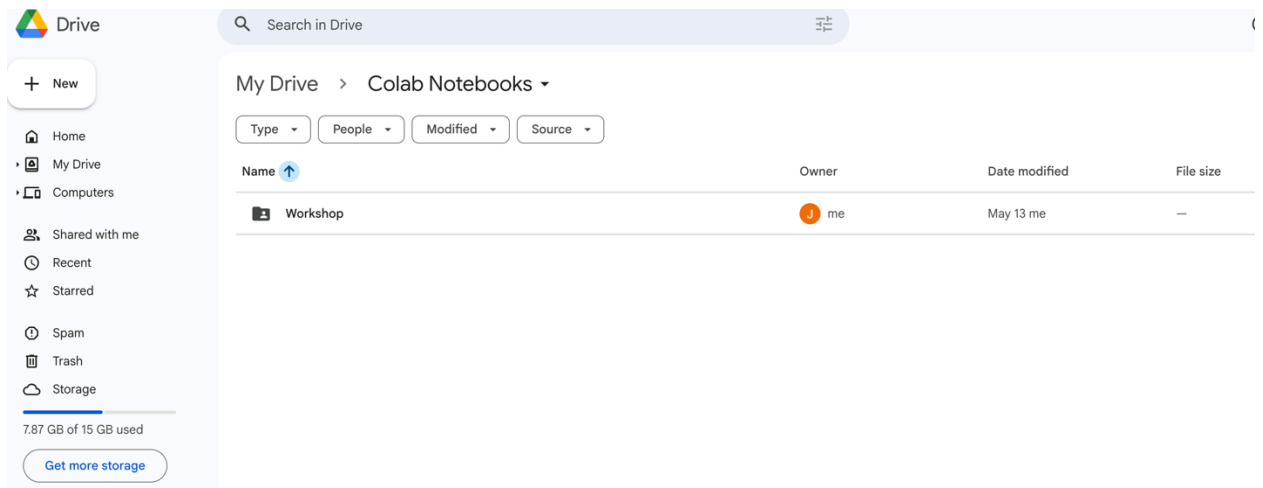
1. Download the GitHub repository for this workshop to your local computer.
2. Log in to Google Colab using your Gmail account:
<https://colab.research.google.com/>
3. Mount your Google Drive in Google Colab.

The screenshot shows the Google Colab web interface. At the top, it says 'Welcome To Colab' and 'Cannot save changes'. Below the menu bar (File, Edit, View, Insert, Runtime, Tools, Help), there are tabs for 'Commands', '+ Code', '+ Text', and buttons for 'Run all' and 'Copy to Drive'. On the left, a 'Files' panel is open, showing a file explorer view. A red box highlights the 'upload' icon (a folder with an arrow) and is labeled 'Step 1'. Another red box highlights the 'Google Drive' icon (a folder with a checkmark) and is labeled 'Step 2'. In the center, a code cell is shown with the following code:

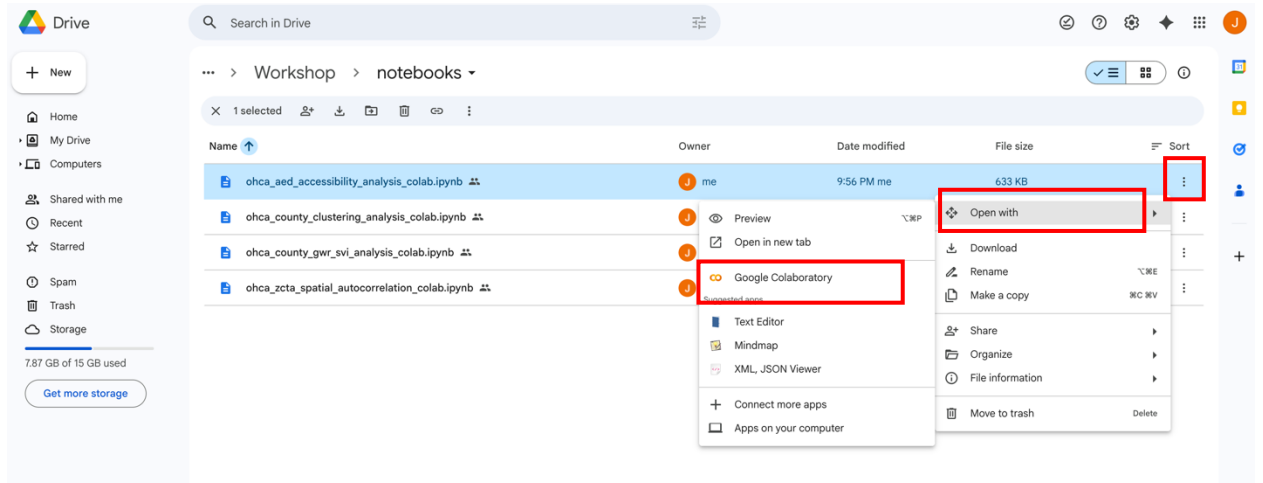
```
from google.colab import drive
drive.mount('/content/drive')
```

 A blue tooltip box is overlaid on the code cell, containing the text 'Run this cell to mount your Google Drive.' and a 'Dismiss' button. Below the code cell, there is a message 'Google Colab is available in VS Code!' with the Colab, heart, and VS Code logos. At the bottom, there is a section titled 'Try the new Google Colab extension for Visual Studio Code. You can get up and running' followed by two bullet points: 'In VS Code, open the **Extensions** view and search for 'Google Colab' to install.' and 'Open the kernel selector by creating or opening any `.ipynb` notebook file in yo clicking the **Select Kernel** button in the top right

4. Open your Google Drive and upload the workshop folder to:
My Drive / Colab Notebooks

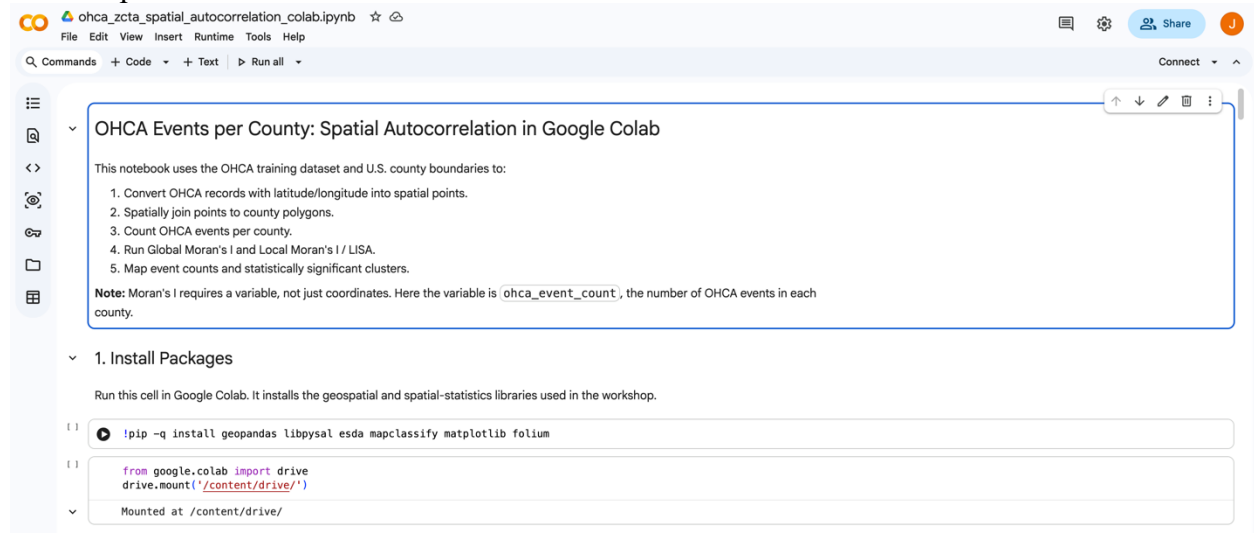


- Open the notebook file (.ipynb) using Google Colaboratory.



- Randomly select one notebook and make sure you can run it successfully. Use Shift + Enter on Windows or Shift + Return on macOS to run each cell. If you are unable to complete this step successfully, no worries—we can work through it together during the

workshop.



In-Workshop Instructions

1. **Spatial Autocorrelation.** ohca_zcta_spatial_autocorrelation_colab.ipynb

Purpose: This notebook test whether OHCA event counts are spatially clustered across Georgia counties. The filename says zcta, but the workflow uses counties.

A. Setup And Libraries

- 1) Installs geospatial and spatial statistics packages.
- 2) Imports geopandas, pandas, matplotlib, libpysal, and esda.
- 3) Imports tools for spatial weights, Moran's I, and Local Moran's I.

B. Load OHCA Points

- 1) Reads OHCA records from the CSV.
- 2) Converts latitude and longitude columns to numeric.
- 3) Drops rows with missing coordinates.
- 4) Creates point geometries.
- 5) Stores OHCA events as a GeoDataFrame in WGS84.

C. Initial Point Map

- 1) Plots all OHCA points.
- 2) Uses the map to check whether the coordinates appear reasonable.

D. Load Georgia Counties

- 1) Reads the county shapefile.
- 2) Reprojects counties to WGS84.
- 3) Filters to Georgia counties.
- 4) Keeps county ID, county name, and geometry.
- 5) Maps county boundaries with OHCA points for alignment checking.

E. Spatial Join

- 1) Joins each OHCA point to the county polygon that contains it.
- 2) Counts how many OHCA points matched a county.
- 3) Counts how many points did not match.
- 4) Previews joined event records with county information.

F. Count OHCA Events Per County

- 1) Groups joined OHCA points by county.
- 2) Counts the number of events in each county.
- 3) Merges those counts back onto county polygons.
- 4) Creates `ohca_event_count`.
- 5) Displays counties with the largest counts.

G. Map Event Counts

- 1) Creates a county choropleth map.
- 2) Colors counties by OHCA event count.
- 3) Uses quantile classification for map classes.
- 4) Helps to visually inspect high-count and low-count counties.

H. Create Spatial Weights

- 1) Builds Queen contiguity weights so neighboring counties are connected.
- 2) Checks whether any counties have no neighbors.
- 3) Uses K-nearest neighbors as a backup if needed.
- 4) Row-standardizes the weights.

I. Global Moran's I

- 1) Uses county OHCA counts as the analysis variable.
- 2) Runs Global Moran's I.
- 3) Reports the Moran's I statistic.
- 4) Reports a p-value.
- 5) Test whether high or low OHCA counts are spatially clustered overall.

J. Local Moran's I / LISA

- 1) Runs Local Moran's I for each county.

- 2) Stores local statistic values and p-values.
- 3) Calculates spatial lag values.
- 4) Classifies counties into cluster types such as high-high, low-low, high-low, low-high, or not significant.
- 5) Counts how many counties fall into each category.

K. LISA Map And Exports

- 1) Maps significant local cluster categories.
- 2) Saves the LISA cluster map.
- 3) Exports county spatial autocorrelation results as CSV.
- 4) Exports mapped results as GeoJSON.

2. **Spatial Heterogeneity.** ohca_county_gwr_svi_analysis_colab.ipynb

Purpose: This notebook focuses on how to model the relationship between county OHCA counts and social vulnerability variables, allowing that relationship to vary spatially using GWR.

A. Setup and Inputs

- 1) Installs geopandas, mgwr, matplotlib, and mapclassify.
- 2) Imports geospatial, data, and modeling libraries.
- 3) Defines paths to OHCA data, county boundaries, and the Georgia SVI table.

B. Create County OHCA Counts

- 1) Loads OHCA event records.
- 2) Cleans latitude and longitude.
- 3) Converts events into point geometries.
- 4) Loads Georgia county polygons.
- 5) Spatially joins OHCA points to counties.
- 6) Counts OHCA cases per county.
- 7) Fills counties with no matched events as zero cases.

C. Load And Prepare SVI Variables

- 1) Reads the SVI county CSV.
- 2) Creates a five-digit county GEOID.
- 3) Selects SVI variables such as age 65+, disability, uninsured, poverty, education, unemployment, no vehicle, and minority status.
- 4) Converts SVI columns to numeric values.
- 5) Checks whether required SVI columns exist.

D. Merge OHCA And SVI Data

- 1) Merges county OHCA counts with county SVI variables.

- 2) Drops counties missing required predictor data.
- 3) Creates the final county modeling GeoDataFrame.

E. Prepare GWR Inputs

- 1) Reprojects counties to EPSG:3857.
- 2) Uses county centroids as model coordinates.
- 3) Sets the outcome variable to `log1p(ohca_cases)`.
- 4) Standardizes SVI predictors using z-scores.
- 5) Creates the predictor matrix for GWR.

F. Fit GWR Model

- 1) Uses bandwidth selection to choose an adaptive neighborhood size.
- 2) Fits a Geographically Weighted Regression model.
- 3) Prints the GWR summary.
- 4) Allows relationships between SVI variables and OHCA counts to vary by county.

G. Save Local Model Results

- 1) Adds local intercepts to each county.
- 2) Adds local coefficients for each SVI predictor.
- 3) Adds local R^2 values.
- 4) Adds filtered local t-values.
- 5) Previews selected coefficient columns.

H. Map GWR Results

- 1) Maps one local SVI coefficient, often `EP_POV150`.
- 2) Shows where that predictor has stronger or weaker local association with OHCA counts.
- 3) Maps local R^2 to show where the model fits better or worse.
- 4) Uses quantile choropleth maps for interpretation.

I. Export Outputs

- 1) Creates an outputs folder.
- 2) Saves county GWR results as CSV.
- 3) Saves county GWR results as GeoJSON.
- 4) Saves the selected coefficient map as PNG.
- 5) Saves the local R^2 map as PNG.

3. **Spatial Clustering.** ohca_county_clustering_analysis_colab.ipynb

Purpose: This notebook focuses how to find geographic clusters of OHCA events in Georgia using point-based clustering methods.

A. Setup and Data Paths

- 1) Installs geospatial, plotting, and clustering packages.
- 2) Mounts Google Drive for Colab use.
- 3) Imports pandas, geopandas, numpy, matplotlib, DBSCAN, and KMeans.
- 4) Sets a local data folder and a Colab data folder.
- 5) Chooses whichever data folder exists.
- 6) Defines paths to the OHCA CSV and county shapefile.

B. Load OHCA Event Points

- 1) Reads the OHCA CSV into a DataFrame.
- 2) Identifies latitude and longitude columns.
- 3) Converts coordinate columns to numeric values.
- 4) Drops records missing latitude or longitude.
- 5) Converts rows into point geometries.
- 6) Creates an OHCA GeoDataFrame in WGS84 coordinates.

C. Keep Georgia Events

- 1) Loads the county shapefile.
- 2) Reprojects counties to WGS84.
- 3) Filters counties to Georgia using STATEFP = '13'.
- 4) Spatially joins OHCA points to Georgia county polygons.
- 5) Keeps only OHCA points that fall inside Georgia counties.

D. Visual Check

- 1) Plots Georgia county boundaries.
- 2) Plots OHCA points on top.
- 3) Uses this map to confirm that the point and county layers align correctly.

E. Prepare For Clustering

- 1) Reprojects OHCA points to EPSG:3857 so distances are measured in meters.
- 2) Extracts x and y coordinates.
- 3) Builds a coordinate matrix for clustering models.

F. DBSCAN Clustering

- 1) Runs DBSCAN with a 5 km search radius.

- 2) Requires at least 10 nearby points to form a cluster.
- 3) Assigns a cluster label to each OHCA point.
- 4) Uses -1 to mark noise points that do not belong to a dense cluster.
- 5) Prints counts for each cluster label.
- 6) Counts the number of non-noise clusters.

G. DBSCAN Mapping and Summary

- 1) Maps DBSCAN clusters.
- 2) Labels noise points separately.
- 3) Summarizes cluster membership by county.
- 4) Helps to identify which counties contribute to each hotspot.

H. KMeans Comparison

- 1) Runs KMeans with 6 clusters.
- 2) Assigns every OHCA point to one of the 6 groups.
- 3) Maps KMeans clusters.
- 4) Compare DBSCAN with KMeans

I. Export Outputs

- 1) Creates an output folder.
- 2) Saves clustered OHCA points as GeoJSON.
- 3) Saves county-cluster summaries as CSV.
- 4) Saves the DBSCAN cluster map as a PNG.

4. **Spatial Accessibility.** ohca_aed_accessibility_analysis_colab.ipynb

Purpose: This notebook focuses on how to measure AED accessibility for OHCA events using simulated AED locations and the Two-Step Floating Catchment Area method.

A. Setup and Data Paths

- 1) Installs geospatial and plotting packages.
- 2) Imports numpy, pandas, geopandas, matplotlib, and Point.
- 3) Sets local and Colab data directories.
- 4) Defines paths to the OHCA CSV and county shapefile.

B. Load OHCA Events

- 1) Reads OHCA records from the CSV.
- 2) Cleans latitude and longitude columns.
- 3) Drops records without valid coordinates.
- 4) Converts each event into a point geometry.
- 5) Creates an OHCA GeoDataFrame.

C. Keep Georgia Events

- 1) Loads county polygons.
- 2) Filters to Georgia counties.
- 3) Spatially joins OHCA events to counties.
- 4) Keeps only OHCA events inside Georgia.

D. Simulate AED Locations

- 1) Dissolves Georgia counties into one state boundary.
- 2) Gets the bounding box around Georgia.
- 3) Randomly generates candidate points.
- 4) Keeps only random points that fall inside Georgia.
- 5) Stops after creating 200 simulated AED points.
- 6) Stores those AEDs in a GeoDataFrame.

E. Map OHCA And AED Locations

- 1) Plots Georgia county boundaries.
- 2) Plots OHCA events as red points.
- 3) Plots simulated AEDs as blue triangle markers.
- 4) Uses the map as a visual check before analysis.

F. Prepare Distances For 2SFCA

- 1) Reprojects OHCA points and AED points to EPSG:3857.
- 2) Extracts x and y coordinates for both layers.
- 3) Calculates the distance from every OHCA point to every AED.
- 4) Creates a distance matrix where rows are OHCA events and columns are AEDs.
- 5) Sets the catchment threshold to 5 miles.
- 6) Converts 5 miles to meters.

G. 2SFCA Step 1: AED Supply Ratios

- 1) Creates a true/false matrix showing whether each OHCA event is within each AED catchment.
- 2) Counts how many OHCA events are within each AED's catchment.
- 3) Assigns each AED a supply value of 1.
- 4) Divides AED supply by nearby OHCA demand.
- 5) Produces R_j , the supply-to-demand ratio for each AED.

H. 2SFCA Step 2: Event Accessibility Scores

- 1) For each OHCA event, identifies AEDs within its catchment.
- 2) Adds up the R_j values of those nearby AEDs.
- 3) Stores this value as the event's A_i 2sfca accessibility score.

- 4) Counts how many AEDs are within each event's catchment.
- 5) Creates a flag for whether each event has at least one AED nearby.

I. Summaries And Maps

- 1) Summarizes event-level accessibility values.
- 2) Groups events by county.
- 3) Calculates county event counts.
- 4) Calculates county mean and median accessibility.
- 5) Calculates the percentage of events with an AED in catchment.
- 6) Maps county mean 2SFCA accessibility.

J. Export Outputs

- 1) Saves simulated AED points as GeoJSON.
- 2) Saves event-level accessibility results as CSV.
- 3) Saves county-level accessibility summaries as CSV.
- 4) Saves the county accessibility map as PNG.